Mashup-Oriented Web API Recommendation via Multi-Model Fusion and Multi-Task Learning

Hao Wu, Yunhao Duan, Kun Yue, and Lei Zhang

Abstract—As the number of Web APIs ever increases, choosing the appropriate APIs for mashup creations becomes more difficult. To tackle this problem, various methods have been proposed to recommend APIs to match requirements of mashups and achieved much success. However, there existed some challenges with feature fusion and utilization, textual requirement understanding, utilization of Mashup categories and compatibility evaluation. Therefore, we propose a neural framework (MTFM) based on multi-model fusion and multi-task learning for Mashup-oriented Web API recommendation. MTFM exploits a semantic component to generate representations of requirements and introduces a feature interaction component to model the feature interaction between mashups and Web APIs. Output features of both components are further fused to predict the candidate APIs, and this enables us to have both the advantages of content-based and collaborative filtering methods. We further introduce mashup category judgment as an auxiliary task, where both tasks are viewed as a multi-label learning problem and jointly optimized with multi-task learning. Also, we have extended MTFM to MTFM++ to take advantage of the metadata and quality features of APIs, and proposed a metric for compatibility evaluation. Experimental results on the ProgrammableWeb dataset show that our methods outperform most popular state-of-the-art methods.

Index Terms—Web API recommendation, mashup, multi-task learning, multi-model fusion, neural network

1 Introduction

With the advent of the Web 2.0 era, Web APIs are gaining more and more attention. Thousands of companies and organizations are adjusting their strategies to attain their value propositions through Web APIs. Particularly, a large number of Web APIs have been released, and an ecosystem of collating and registering APIs has also appeared in recent years [1]. We can find more than 24,000 APIs in ProgrammableWeb which is the largest online API registry. These APIs cover nearly 500 frequently-used application types, such as social, video, media, and so on. Through the standard interfaces, any compatible device can easily call the existing Web APIs in an open network environment. The advantages of Web APIs give birth to a new technique of application development, namely Mashup [2]. Mashup allows users to integrate existing Web resources to create composite Web applications and react to complex business needs. Compared with the need to write code from scratch, developers do not prescribe to reinvent the wheel for each new application [3]. Instead, they can focus on the goal of achieving the unique needs of the application, which greatly releases the developer's creativity [2], [4]. As showed in Fig. 1, Vizlingo is a popular mobile application that animates each word of a text message or social media post with an user-generated video clip. To provide users with comprehensive social data, it integrates a few different APIs, including YouTube, Facebook, and Twitter.[1]

Generally, developers can browse and evaluate potentially useful Web APIs from the repositories, then exploit mashup tools "centrally" to include selected Web APIs their applications quickly [4]. The number of existing Web APIs, however, has been increasingly rapidly in the past several years. It is impractical, if not impossible, for developers to inspect a vast number of candidate Web APIs, especially developers with limited experience in mashup development. Similar to code recommendation [5], third-party library recommendation [6] and web service recommendation [7], recommendation techniques can also be employed to recommend suitable Web APIs to developers for mashup development [8]. Web API recommendation can be included during the mashup development cycle, after requirement analysis and specification and before mashup composition, as illustrated in Fig. 2. It will significantly reduce developers' time and efforts devoted to exploring and discovering useful Web APIs.

A variety of methods have served to API recommendations. These methods can be summarized as collaborative filtering (CF) based methods [9], [10], [11] and content-based methods [12], [13], [14], [15], [16], [17]. Given the requirement of a mashup, CF-based methods suggest APIs by identifying existing mashups that are highly similar to the requirement and then ranking the candidates of Web APIs through aggregating the relevant score between the candidates and the selected mashups. Content-based methods focus on mining the relationship between requirements and Web APIs, and various technologies, e.g., topic

- *Hao Wu, Yunhao Duan, and Kun Yue are with the School of Information Science and Engineering, Yunnan University, Kunming 650091, China. E-mail: {haowu, kyue}@ynu.edu.cn, yhduan@mail.ynu.edu.cn.*
- *Lei Zhang is with the School of Electrical and Automation Engineering, Nanjing Normal University, Nanjing 210046, China. E-mail: leizhang@njnu.edu.cn.*

Manuscript received 6 September 2020; revised 20 June 2021; accepted 18 July 2021. Date of publication 21 July 2021; date of current version 9 December 2022. (Corresponding author: Kun Yue.)
Digital Object Identifier no. 10.1109/TSC.2021.3098756

1. https://www.programmableWeb.com/mashup/vizlingo

1939-1374 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

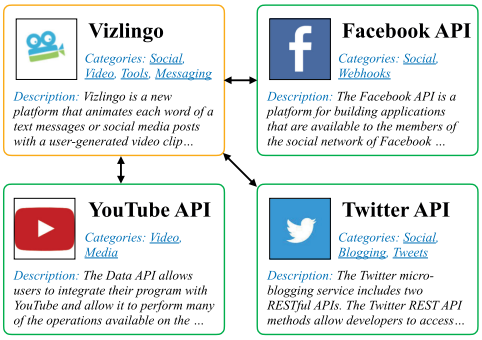

Fig. 1. The mashup example of Vizlingo.

modeling [12], [14], [18], link analysis [15], [19], deep learning [20], [21], [22], and various features (texts, tags, topics, and popularity) are exploited to enhance the accuracy of recommendations [23], [24], [25], [26]. Even though many methods have achieved much success, there still exist the following issues that prevent them from being applicable to practices:

a) The methods dominated by collaborative filtering and content analysis are still weak in feature fusion and utilization when designing recommendation models. CF-based methods can share the experience of similar mashups, and can also achieve certain results in the case of sparse data, but do not give enough attention to the matching between Web APIs and mashup requirements [9], [10], [11]. In contrast, content-based approaches heavily rely on description documents to estimate the relevance between requirements and Web APIs. However, the recommendation results would be unsatisfactory when it does not give sufficient information in the textual requirements [19]. How to harmonize diverse algorithmic mechanisms to overcome weakness in a single model and improve the effectiveness of recommendations needs to be continuously explored.

b) They often require users to input a set of keywords for characterizing the expected functions of mashup. However, the requirement is usually an informal expression in the step of discovery and selection. It is quite unrealistic for users to choose keywords accurately to articulate their requirements. To overcome these shortcomings, it is necessary to introduce natural language processing technology to accurately summarize those freestyle documents of requirement descriptions and align them to standard categories/tags [20], [21], [22]. At the same time, how to use the category/tag information of mashup to optimize recommendation models emerges as a new research issue.

c) Mashup composition needs to consider the compatibility of candidate APIs. However, the traditional compatibility evaluation needs to know the detailed knowledge of the input and output of APIs. The reality is not optimistic because a large number of APIs do not possess detailed semantic annotations for input and output. Historical mashups imply that those APIs participated in the same mashup are compatible with each other [27], but how to measure the compatibility of candidate APIs based on these historical information remains to be further explored.

To overcome the limitations of existing recommendation models, we propose a novel technological framework in this paper. Requirements of mashup development will be

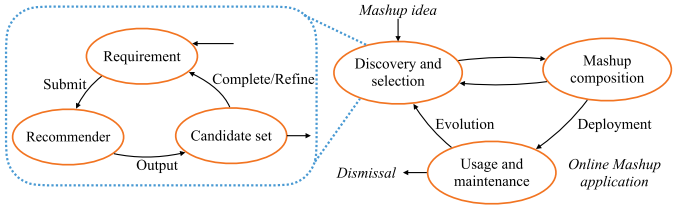

Fig. 2. The activity of Web API Recommendation in the life-cycle of *light development process* coined by Daniel *et al.* [4].

described in the natural text, which practically provides developers with an easy manner to represent their requirements. Considering that deep learning technology has made great progress in natural language processing [28], we utilize a semantic component (a convolutional neural network) to extract the semantic features of the requirement and express it as a vector representation. These representation vectors will input a feature interaction component to extract interactive features with all the embeddings of Web APIs. Then, the representative vector of the requirement will concatenate with the interactive feature vectors to realize the fusion of both features and models. This process is called multi-model fusion. The advantage of this treatment is that it allows our framework to have both the benefits of content-based and CF-based methods. The semantic component can better capture the characteristics of requirements through a deep understanding of the content, while the feature interaction component can provide us with the ability of memory-based collaborative filtering. Besides, metadata and quality features of Web APIs can be employed to enhance input and further improve the framework.

As for the task of Web API recommendation, we turn it into a multi-label classification problem. By predicting the relevance score between mashup requirements and all APIs, we can generate the recommendation list by ranking the candidate APIs with relevance. At the same time, we add a category judgment task to enable our framework to have multi-task learning ability considering the importance of categories for Web APIs and mashups: categories can be regarded as precise summaries of API functions, so they carry richer information; categories have been recognized as successful in organizing and sharing Web APIs in ProgrammableWeb. Multi-task learning is an inductive transfer method that takes full advantage of the specific domain information implicit in the training signals of multiple related tasks. In the process of backward propagation, multi-task learning allows features in a shared hidden layer dedicated to a certain task to be used for other tasks, learning features that are often not available in a single-task learning network apply to several different tasks. By determining the category of mashups, the training signal information implicit in the category judgment task can be used as an inductive bias to improve the ability of API recommendation. Also, it enables us to simultaneously recommend APIs and suggest categories of mashups. The main contributions of this article can be summarized as following:

- We have proposed a multi-task fusion model (MTFM) for mashup-oriented Web API recommendation. MTFM combines the advantages of both CF-based and content-based methods, implements

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

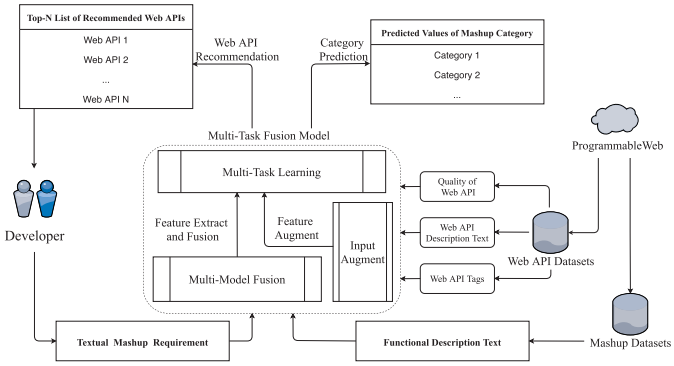

Fig. 3. The process of Mashup-oriented Web API recommendation.

mashup category prediction and API recommendation simultaneously given the textual requirement.

- We have extended MTFM to MTFM++ which can use the metadata and quality features of Web APIs to achieve better prediction results.
- We have proposed a metric to quantify the compatibility of recommendation results based on the historical records of APIs participating in mashups.
- Intensive experiments are conducted on the ProgrammableWeb dataset and the results show that our proposed methods achieve better accuracy and compatibility than many SOTA methods.[2]

The resting paper is structured as follows : Section 2 formulates the problem of Mashup-oriented Web API recommendation and presents the technical details of our solution. Section 3 conducts experiments and results analysis. Section 4 presents a review of related work. Finally, we summarize the current work and think about the future work.

2 Method Overview

In the actual scenario, there are three steps in the process of mashup-oriented Web API recommendation: First, we need to build a recommendation model; Second, the recommendation model will be trained against the usage data of Web APIs and Mashups gathered from service providers; after that, developers or users submit their requirements described in natural language, and the recommender system will generate a candidate collection of Web APIs, at the same time, propose feasible tags/categories. The process is demonstrated in Fig. 3. In this section, we will discuss our solution in detail.

2.1 Problem Definition

- A *mashup* $m = <D_m, C_m, T_m, X>, m \in M$ consists of a textual description D_m to introduce its functions, a set of categories/tags T_m to index it, a set of API components C_m to participate in the mashup, and a set of meta-elements X to prepare for composition. X contains data pipes, view ports that can host and render components, layout that specifies the graphical arrangement of components according to Imran *et al.* [29]. Note that X depends different mashup tools, so it is optional in our work.
- A Web API $a = <D_a, T_a, q_a, IP, OP>, a \in A$ consists of a textual description D_a to introduce its functions, a set of categories/tags T_a to index it, a vector q_a to quantify its attributes of quality, a set of input ports IP carry data into the API, a set of output ports OP carry data generated by the API. Each API has at least either an input or an output port [29].
- *Requirement* is a textual document r to represent a user requirement for mashup creation. We do not set a formal template for the requirement, and just assume that it is unstructured text.
- *Recommendation Model* is defined as a non-linear parameterized function $\hat{y} = f(\theta, r, A)$. After determining the values of model parameters θ, we can use the function f to estimate the relevance scores between r and A, and then rank all Web APIs according to $\hat{y} \subseteq \{0, 1\}^{|A|}$.

2.2 Preprocessing

The text descriptions of Mashups/Requirements and Web APIs usually contain noise information and cannot be used directly for model learning. To reserve the important information in the texts, the following processes are usually required.

- *Text Filtering:* Invalid words such as tags, punctuation marks, non-characters, etc, are usually unmeaningful, we use regular expressions to filter them out. Also, stop words are useless for semantic analysis and have to be eliminated.
- *Abbreviation Replacement:* For example, 'can't' is an abbreviation for 'can not', which is equivalent in human eyes, but the machine does not think so. Consequently, we replace such abbreviations with complete spelling.
- *Lemmatization:* Lemmatization is to remove the affix of the word and extract the main part of the word. Usually, the extracted word will be the word in the dictionary. For example, the word "cars" is reduced to the word "car", and the word "ate" is "eat".
- *Word Vectorization:* Collect all the words and number the words to build a vocabulary. All words in the vocabulary are independent and have a one-to-one index. After getting the vocabulary, we use Glove.6B.300d pre-trained word vectors[3] to initialize the embeddings of existing words, and randomly initialize those cold-start words.

For category information, we binarize it into a one-hot vector with a length of the number of categories, the value involved in mashup is 1, and the rest is 0.

2.3 Multi-Task Fusion Model (MTFM)

2.3.1 Semantic Component (SC)

As for the textual description of Mashups, Web APIs, and requirements, it contains richer information than other metadata. Therefore, text modeling or representation technique is essential in content-based recommendation methods. However, traditional modeling approaches are built on the '*bag of words* (BoW)' model. The word order and

2. The code is available at https://github.com/whale-ynu/MTFM

3. https://nlp.stanford.edu/projects/glove/

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

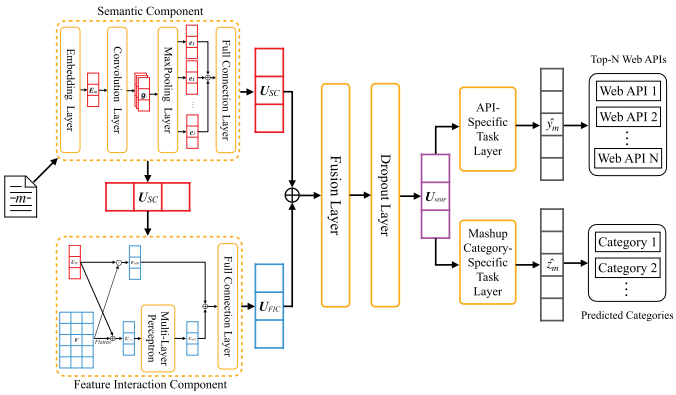

Fig. 4. Multi-task fusion model.

grammar of the text are ignored [30], and the selection of features needs human intervention in the BoW model.

With the further prosperity of deep learning, massive attention has been paid to exploring text modeling with deep neural networks [28]. As neural network models have the capability of feature selection, they can make use of word contexts to achieve better semantic representations, especially for convolutional neural network (CNN) [31]. On the one hand, CNN is mature and efficient, which can quickly process high-dimensional data by sharing convolution kernels; on the other hand, CNN can automatically extract features, which enable better prediction results through a simple strategy of data augmentation. This provides users with technical support for incremental service discovery. It can be proved by an example in a later section where the recommendation effect can be improved by enhancing the textual description of the Mashup requirement. For these reasons, we use the CNN as the basic component for feature extraction in textual documents (as showed in Fig. 4).

The core idea of the convolutional neural network (CNN) is to capture local features from sliding windows composed of several words. As for the semantic component, it can automatically combine and filter features to obtain semantic information at different levels of abstraction. Let $\mathbf{E}_m = lookup_table(m)$ represent the word embedding matrix corresponding to the text of mashup/requirement m where $lookup_table$ is a primitive operation for embedding, and the dimension of word vectors d is determined by a pre-trained language model, such as GloVe and Word2Vector.

A convolution kernel in the convolution layer, $\mathbf{G}_i \in \mathbb{R}^{ws_j \times d}$ is first sliding on the word window with the size of ws_j to perform convolution operation

$$g_i = relu(\mathbf{G}_i \mathbf{E}_m + \mathbf{b}_i), \tag{1}$$

where b_i is the bias term, and $relu$ (Rectified Linear Unit) is used as the activation function. After the convolution operation, one feature map g_i is obtained for each convolution kernel. And then, to extract important features, we perform *Max-Pooling* operations on all feature maps to simplify redundancy details of features, and then concatenate the features that have been pooled as follows,

$$e_j = [MaxPooling(g_i); \ldots; MaxPooling(g_I)], \tag{2}$$

where I is the number of convolution kernels. Generally, we need windows of different sizes to enhance feature extraction. In this case, the results of multiple windows will also be concatenated together by $e = [e_1; \ldots; e_j; \ldots; e_J]$, where e_j corresponds to the results of the word window with a size of ws_j. Finally, we feed $\mathbf{e}$ into a full-connection layer to achieve further non-linear transformation as follows:

$$U_{SC} = sigmoid(\mathbf{W}_0 e + \mathbf{b}_0), \tag{3}$$

where $\mathbf{W}_0$ is the weight matrix, and $\mathbf{b}_0$ is the bias term, *sigmoid* is the activation function. To express the above-mentioned transformation process conveniently, we rewrite it as $U_{SC} = CNN(\theta, m)$. $CNN(\theta_0, m)$ is the parameterized function to encode the text of m and $\theta = \{\mathbf{E}_*, \mathbf{G}_*, \mathbf{W}_0, \mathbf{b}_*\}$.

2.3.2 Feature Interaction Component(FIC)

Factorization technologies, such as matrix factorization (MF) and factorization machine (FM), are widely used in the field of recommendation systems [32]. The basic principle among them is to decompose a sparse user-item interaction matrix R into the product of two dense low-rank (k) matrices U, V: $R \approx UV^T$, and then the two matrices can be used to embed the users and the items respectively. It can recover from the difficulty of user/item embedding representations caused by data sparsity in collaborative filtering. At the same time, the representation of users and items can capture the neighborhood effect in the same semantic space, that is, similar users or items will project comparable regions in the space due to the clustering effect of matrix factorization.

In our scenario, the feature interaction process expands over the invocation matrix between mashup and Web APIs, and thus both mashups and Web APIs can obtain embedding representations in the same space. Let $U_{SC} \in \mathbb{R}^k$ be the embedding vector of mashup m, and $\mathbf{V} \in \mathbb{R}^{|A| \times k}$ be the embedding vectors of all Web APIs. It is worth noting that $\mathbf{V}$ can also be initialized by processing various metadata of the APIs, but we leave it for a later section. We first generate the interaction vectors of mashup m with Web APIs as follows:

$$U_{MM} = U_{SC} \cdot \mathbf{V}^T. \tag{4}$$

where $\cdot$ is the element-wise production. This kind of linear operation can remember those common and highly frequent interaction patterns in historical data. However, it is sometimes difficult to model those rare but useful interactions of the features of Mashup and Web APIs. To solve this problem, the embedding vectors of mashup and Web APIs are concatenated together by

$$U_{Concat} = [U_{SC}; Flatten(\mathbf{V})], \tag{5}$$

where $Flatten(V)$ is a flatten operation to transform V into a 1-dimensional vector. After that, U_{Concat} will be fed into a multi-layer perceptron (MLP) to learn high-order interaction of the features of Mashup and Web APIs as follows:

$$U_{MLP} = \delta(\mathbf{W}_z(\ldots \delta(\mathbf{W}_1 U_{Concat} + \mathbf{b}_1) \ldots + \mathbf{b}_z), \tag{6}$$

where $\mathbf{W}_z$ and $\mathbf{b}_z$ represent weight and bias vectors in the zth layer of MLP, δ is an activation function. The advantage of MLP is that it can learn interactive features at different

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

levels of abstraction. With the increase of the number of layers, the receptive field of each neuron becomes larger relative to the previous layer, so it can provide global semantic and abstract details, which is difficult to do in a shallow layer and linear operation.

After concatenating the two feature vectors U_{MM} and U_{MLP}, we get the final output of Feature Interaction Component (see Fig. 4)

$$U_{FIC} = [U_{MM}; U_{MLP}]. \tag{7}$$

Here, we use *Concat* operation instead of other operations for feature fusion. The main consideration is that the best vector dimension characterizing low-order interaction features may be different from that of high-order interaction features. It's easier to handle this situation with the *Concat* operation. FIC combines the linearity of MF and the non-linearity of MLP for modeling Mashup–API latent structures, and thus it can capture low-order and high-order interactive features at the same time.

2.3.3 Multi-Model Fusion & Multi-Task Learning

The strategy of multi-model fusion is to integrate different types of features or models to improve the performance of machine learning models, in particular, neural network models, and make up for the heterogeneity of different models. So far, we have introduced the semantic component and the feature interaction component to generate features from different levels and angles of view for the prediction tasks. Here, multi-model fusion will be adopted to aggregate the obtained features as showed in Fig. 4. For our case, we adopt the strategy of early fusion which is immediately followed by downstream tasks [33]. Let $\mathbf{W}_{TL}$ and $\mathbf{b}_{TL}$ be the weight matrix and the bias vector in a specific task layer, formally, we have

$$\hat{y}_m = sigmoid(\mathbf{W}_{TL} U_{MMF} + \mathbf{b}_{TL}), \tag{8}$$

where $U_{MMF} = [U_{SC}; U_{FIC}]$. Here, to suppress the overfitting of our proposed model, the trick of *dropout* is adopted.

Multi-task learning is an inductive transfer method that makes full use of the specific domain information implicit in the training signals of multiple related tasks. In the process of backward propagation, multi-task learning allows features in a shared hidden layer dedicated to a certain task to be used by other tasks, and thus enable the effects of implicit data augmentation and co-regularization which seriously benefit the model training of a deep neural network [34]. To make our model have the capability of multi-task learning, we introduce two specific task layers.

As for the main task (MT) of Web API recommendation, we transform it into a multi-label classification problem [35] which is further decomposed into multiple independent binary classification problems (one per API). Here, each API in the collection A is treated as an individual label. The objective is to train a classifier that can automatically tag a mashup m with the most relevant subset of A. To train our model through the main task, we adopt Binary Cross-Entropy Loss (BCELoss), which is defined as follows:

$$\begin{aligned}\mathcal{L}_{MT}(\hat{y}_m, y_m) = -\frac{1}{|A|}\sum_{a\in A}(&y_m[a]\log(\hat{y}_m[a]) \\ &+ (1 - y_m[a])\log(1 - \hat{y}_m[a])),\end{aligned} \tag{9}$$

where $y_m[a]$ corresponds to the target value which indicates whether or not the ath Web API matches the requirement of mashup m ($y_m[a] = 1$ if m invoked a; otherwise $y_m[a] = 0$), and $\hat{y}_m[a]$ is the predicted value corresponding to $y_m[a]$. After model training, we can generate the recommendation list of the candidate APIs for a requirement m by predicting the relevance scores $\hat{y}_m \subseteq \{0,1\}^{|A|}$ and ranking all APIs according to these scores.

At the same time, we add a category judgment task to enable our framework to have multi-task learning ability considering the importance of categories for Web APIs and mashups. Similar to the main task, we also turn this auxiliary task (AT) into multiple independent binary classification problems (one per category $c \in C$) and adopt the BCELoss ($\mathcal{L}_{AT}$) to optimize the model parameters.

$$\begin{aligned}\mathcal{L}_{AT}(\hat{z}_m, z_m) = -\frac{1}{|C|}\sum_{c\in C}(&z_m[c]\log(\hat{z}_m[c]) \\ &+ (1 - z_m[c])\log(1 - \hat{z}_m[c])).\end{aligned} \tag{10}$$

By aggregating the loss functions of the two tasks and imposing regularization constraints, we can obtain the objective function of our model as follows:

$$\min \frac{1}{|M|}\sum_{m\in M}(\mathcal{L}_{MT} + \mathcal{L}_{AT}) + \lambda||\theta'||_2^2, \tag{11}$$

where $\theta' = \theta \cup \{\mathbf{W}_1, \mathbf{b}_1, \ldots, \mathbf{W}_z, \mathbf{b}_z, \mathbf{V}\}$ is the model parameters, and $\lambda||\theta'||_2^2$ is a L_2 regularization term used to inhibit the model from over-fitting.

Because that our model is built on deep neural networks, we can utilize an automatic derivation and optimization framework (e.g., TensorFlow[4] and PyTorch[5]) to implement the model optimization. For each layer, model parameters can be updated with the back-propagation algorithm which iteratively computes gradients using the chain rule. Many gradient descent optimization algorithms, such as Adam, Adagrad, Adadelta, and RMSprop can be used. Moreover, a *mini-batch* gradient descent, which processes a small subset of the training set in each iteration, is necessary to speed up training on the large-scale datasets.

2.4 MTFM++

In this section, we will extend MTFM to MTFM++ by adding neural component and layer to exploit the metadata and quality features of Web APIs (see Fig. 5).

2.4.1 Exploiting Text and Tags of Web API

There are many metadata about Web APIs, such as textual documents, tags/categories, service providers, and so on. Intuitively, this information can be used as another kind of

4. https://www.tensorflow.org/
5. https://pytorch.org/

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

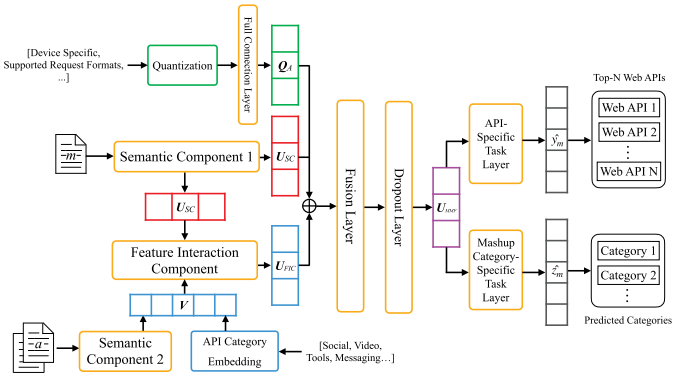

Fig. 5. The model of MTFM++.

TABLE 1
The Quality Features of Web APIs

First-Level	Second Level	Quantified Value
Functionality	Device Specific, Supported Request Formats, Supported Response Formats, SSL Support, Authentication Mode	1 or 0, #SupportedFormats/#All SupportedFormats, #SupportedFormats/#All SupportedFormats, 1 or 0, #AuthenModes/#All AuthenModes
Reliability	Version	Version/ MaxVersion
Usability	Articles, Developers, Libraries, Mashups, SourceCodes, SDKs	#Articles/#MaxArticles, #Developers/#MaxDevelopers, #Libraries/#MaxLibraries, #Mashups/#MaxMashups, #SourceCodes/#MaxSourceCodes, #SDKs/#MaxSDKs

model input to enhance the embeddings of Web APIs. To achieve this goal, we first add the second semantic component as a precursor layer of the embedding of Web API (a.k.a $\mathbf{V}$). The description text of Web API a will be then fed into the component after preprocessing (refer to Section 2.2). Formally, we have $\mathbf{V}_1^a = CNN(\theta_2, a)$ where θ_2 is the component parameters. Second, we set a separate embedding layer for mapping the tags/categories of Web API. Formally, we have $\mathbf{V}_2^a = lookup_table(T_a)$ where T_a is the set of tags/categories of a. Finally, the two vectors will be concatenated for every Web API, we then have $\mathbf{V} = \{[\mathbf{V}_1^a; \mathbf{V}_2^a] | a \in A\}$.

2.4.2 Exploiting Quality Features of Web API

Attributes of Quality of Services (QoS) usually relate to the non-functionality of Web APIs, such as availability, reliability, and response time. However, these QoS values are difficult to obtain with Web APIs as their internal details are typically hidden[36]. Instead, we build our quality features based on the quality model which was first proposed by Cappiello *et al.* [36] and later applied by Fletcher [26]. Details of quality features are summarized in Table 1.

The *functionality* of a Web API can be refined by considering its *interoperability*, *compliance*, and *security* level [36]. *Interoperability* depends on whether a Web API is *device specific* or not. *Compliance* of a Web API is quantified by the ratio of its number of *message data formats* to the total number of all standard data formats [36]. The *security* is related to two main aspects: SSL support and authentication mechanisms [36]. *Reliability* of a Web API is directly measured concerning its version. A Web API's *usability* is evaluated in terms of the number of articles, developers, libraries, sample source codes. SDKs, and any other kind of documentation [26].

Compared with textual features, quality features need to be quantified before inputting data. For all second-level quality features, they are quantified according to Table 1 and their values are packed into a vector q_a to represent the original quality of a. Then, a fully-connected layer is added for the preliminary feature selection: $Q_a = tanh(\mathbf{W}_q q_a + \mathbf{b}_q)$. Finally, all quality vectors $Q_A = \{Q_a | a \in A\}$ will concatenated with U_{SC} and U_{FIC} to make fusion: $U_{MMF} = [U_{SC}; U_{FIC}; Q_A]$.

TABLE 2
Statistics of the Experimental Dataset

Statistics	Value
# APIs	1,647
# Mashups	8,217
# Categories	499
# API Invocations by Mashups	17,182
# APIs per Mashup	2.091
# Categories per Mashup	2.447
# Categories per API	2.271
# Words in description per Mashup	17.019
# Words in description per API	44.296
# Words in description	17,133

3 Experiments and Analysis

In this section, we will conduct a series of experiments to evaluate our proposed methods and answer the four research questions. Also, case analysis will be done to make an intuitive evaluation.

- RQ1 Does the semantic component and the feature interaction component contribute to the performance of MTFM?
- RQ2 Does multi-task learning contribute to the performance of MTFM?
- RQ3 Does the additional information of Web APIs contribute to the performance of MTFM?
- RQ4 Are the proposed methods competitive compared with SOTA methods?

3.1 Datasets

As of December 2019, we crawled 22642 Web APIs and 8484 Mashups from ProgrammableWeb.[6] For each mashup, we obtain the four kinds of metadata: name, description, category, and API invocation information. For each API, we obtain its three kinds of metadata: name, description, and category. Owing to the lack of one or more of the above corresponding primary metadata, we removed 20,995 invalid Mashups and 20,995 invalid APIs. Finally, our dataset includes 499 categories of applications, 8,217 Mashups, 1,647 APIs which were invoked by Mashups at least once. Detailed statistics of the dataset are presented in Table 2. To conduct performance evaluation, we use the hold-out method to randomly divide the dataset into a training set, a

6. www.programmableweb.com

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

validation set, and a testing set, which includes 6,573, 822, and 822 Mashups respectively.

3.2 Evaluation Metrics

In our scenario, users can request Web APIs by writing and submitting their queries, and then our recommendation system (RS) gives them back the top-N most relevant results. Therefore, our recommendation system can be regarded as ad-hoc information retrieval (IR) system. There are many protocols on the evaluation of IR and RS systems, such as offline evaluation, user surveys, and online evaluation [37]. Due to the high cost of user surveys and online assessments, we adopt four offline and widely-used indicators of accuracy: Precision@N, Recall@N, MAP@N, and NDCG@N. Precision@N and Recall@N are single-value metrics and do not consider the ranking order of items, while MAP@N and NDCG@N are order-aware metrics where positions are discounted. Moreover, users usually pay more attention to the top items in a recommendation list, so the length of the result list is set to $N \leq 30$.

Precision@N is the ratio of the number of real hits in the recommended APIs to the number of recommended APIs

$$Precision@N = \frac{|realAPIs \cap topNAPIs|}{|topNAPIs|}. \tag{12}$$

Recall@N refers to the ratio of the number of real hits in the recommended APIs to the number of all real APIs

$$Recall@N = \frac{|realAPIs \cap topNAPIs|}{|realAPIs|}. \tag{13}$$

Normalized Discounted Cummulative Gain (NDCG) is a measure of accuracy from IR

$$DCG@N = rel_1 + \sum_{i=2}^{N} \frac{rel_i}{log_2(i+1)}, \tag{14}$$

$$IDCG@N = \sum_{i=1}^{|realAPIs|} \frac{2^{rel_i}}{log_2(i+1)}, \tag{15}$$

$$NDCG@N = \frac{DCG@N}{IDCG@N}. \tag{16}$$

$rel_i = 1/0$ indicates whether the ith Web API is truly relevant to the current mashup or not. DCG is accumulated from the top of the result list to the bottom, with the gain of each result discounted at lower ranks. IDCG@N indicates the ideal DCG at N. Similar to NDCG@N, Mean Average Precision (MAP) is also widely used in IR and RS

$$MAP@N = \frac{1}{|M|} \sum_{m \in M} AP_m@N, \tag{17}$$

where $AP@N$ for the recommendation list of mashup m is given by

$$AP@N = \frac{\sum_{i=1}^{N} rel_i * Precision@i}{\sum_{i=1}^{N} rel_i}. \tag{18}$$

Here, AP is the average precision of all hit positions in the recommendation list.

Compatibility problems can arise when interconnecting different APIs to create a mashup. A higher compatibility of the candidate set of APIs means a higher success rate on developing the mashup. Traditionally, compatibility is quantified by the matching degree between the input and output of two APIs [27]. However, such a manner can be no certainty, as a large number of APIs may not have detailed semantic annotations for input and output. Historical record of mashups implies that those APIs participated in the same mashup are most likely compatible with each other [27], so we can measure the compatibility of candidate APIs based on their co-invocation patterns. Let G be the co-invocation graph of APIs, then if APIs i and j were co-invoked, there exists an edge between the two. Denote $d(i,j) \geq 1$ the shortest distance of i and j against G, the compatibility of i and j is defined as: $cpb(i,j) = e^{1-d(i,j)}$, and CPB@N for the recommendation list of m is given by

$$CPB@N = \frac{2}{N(N-1)} \sum_{i=1}^{N} \sum_{j=i+1}^{N} cpb(i,j). \tag{19}$$

If j is compatible with i, k respectively, there will be $cpb(i,j) = 1$ and $cpb(j,k) = 1$, then $cpb(i,k) = e^{-1}$. Obviously, $cpb(i,j) \in (0,1]$, so CPB@N is also in $(0,1]$ after normalized by the number of all pairs.

3.3 Evaluation Methods

Currently, there are no universal benchmarks for the recommendations of Web APIs. Therefore, we use a variety of representative recommendation methods as baselines to verify the effectiveness of our proposed methods.

Popularity-Based Model (Pop). The widely-used *Pop* model directly suggests highly popular items to users. For API recommendations, we treat the popularity of API as its number of category-aware invocations, that is, APIs with high category-aware frequency are ranked in front of the recommendation list.

Collaborative Filtering (CF). Mashup-based *CF* is a classic recommendation algorithm [8]. It calculates the similarity $sim(i,j)$ between the target mashup and the rest of the mashups [38], and then recommends the APIs involved in the top-similar mashups against the score function

$$score(m_i, a) = \sum_{m_j \in M, m_j \neq m_i} sim(m_i, m_j) * r(m_j, a), \tag{20}$$

where $r(m_j, a) = 1/0$ indicates whether or not the API a is relevant to the mashup m_j.

Neural Collaborative Filtering (NCF). NCF is a classic neural model in recommender systems [39]. For our scenario, we initialize the embedding of mashup using our semantic component with the Glove.6B.300d pre-trained word vectors and train the model through implicit feedback of the invocation between mashups and APIs.

Bayesian Personalized Ranking (BPR). BPR is widely-used for predicting user preference on different items [40]. Given a pair of a_i and a_j, if mashup m calls a_i, then we get a triplet (m, a_i, a_j), which means that a_i is ranked higher than a_j for

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Performance Comparison of Ablation Models w.r.t MTFM on the Task of Web API Recommendation

Methods	NDCG@5	NDCG@10	NDCG@30	Recall@5	Recall@10	Recall@30	CPB@5	CPB@10	CPB@30
STFM	0.5964	0.6122	0.6167	0.4985	0.5670	0.6741	0.8763	0.8553	0.7454
MTM	0.6208	0.6309	0.6350	0.5459	0.6003	0.7107	0.8803	0.8513	0.7460
MTFM	**0.6732**	**0.6767**	**0.6775**	**0.5867**	**0.6432**	**0.7502**	**0.9236**	**0.8725**	**0.7541**

m. We initialize the embedding of mashup the same as NCF.

LSTM Model (LSTM). LSTM is widely used as a semantic component to build classifiers [41]. Here, we use the Bi-directional LSTM with Attention to extract the feature vector of the mashup description, which is similar to the strategy in [22]. The model is optimized using the Multi-Label Soft Margin Loss function [41]. Also, we use the Glove.6B.300d pre-trained word vectors.

Service Profile Reconstruction Model(SPR). Zhong *et al.* [18] use the author-topic model to model the relationship between the mashup description and the APIs, and then make API recommendations against topical structure analysis of the mashup description.

Random Walks With Restart (RWR). Wang *et al.* [19] design a concise schema of the knowledge graph to encode the mashup-specific contexts and model the mashup requirement with tag/category entities. The RWR algorithm is performed with preferring those entities and then ranking candidate APIs with their relevance scores.

Functional and Contextual Attention-Based LSTM Model (FC-LSTM). Shi *et al.* [22] propose a LSTM based model with a functional attention mechanism and a contextual attention mechanism to help select the appropriate services. For the sake of fairness, we omit the text enhancement stategy [22] and initialize the word embedding with the Glove.6B.300d pre-trained word vectors.

Multi-Task Fusion Model (MTFM & MTFM++). In the feature interaction component, the embedding dimension of both mashups and APIs is set to 8, and the number of layers in the Multi-Layer Perceptron is set to 2. In the semantic component, the interception length of texts is set to 50, the dimension of the pre-training word vector is fixed at 300, the context window is set with the size of {2, 3, 4, 5} and each of which has 64 convolution kernels. Adam is used as the optimizer and the learning rate is set to $1e-3$. Also, an early-stopping strategy is adopted during training.

3.4 Ablation Study (RQ3 & RQ4)

Ablation study is widely used to detect the importance of components in machine learning models, especially in complex neural networks. For our MTFM, there are two components worth studying and analyzing. The first one is the FIC which provides collaborative filtering capability for MTFM. By removing it from MTFM, we obtained an ablation model as the Multi-Task Model (MTM), which conducts both tasks of API recommendation and mashup category judgment only with the SC. The second one is the auxiliary task in the multitasking module. By removing the task of mashup category judgment in MTFM, we obtain another ablation model of Single-Task Fusion Model (STFM), which predicts the candidate APIs with the outputs of both the SC and the FIC.

Table 3 shows the performance comparison of MTM, STFM, and MTFM on the task of API recommendation. Both strategies of multi-model fusion and multi-task learning are successful in improving the accuracy. By comparing STFM with MTFM, we find that adding the auxiliary task can bring 6.4-6.1 percent of the revenue in NDCG@ {10-30}, and 7.5-7.6 percent of the revenue in Recall@ {10-30}. By comparing MTM with MTFM, we find that it reduces the accuracy of recommendations by 4.5-4.3 percent in NDCG@ {10-30}, and 4.3-4.0 percent in Recall@{10-30} without the FIC. The same results can be observed in the task of mashup category judgment, according to Table 4, where the performances of MTM and MTFM are also compared. Furthermore, we can intuitively reveal that the strategy of multi-task learning has more influence on improving the accuracy of recommendations when comparing MTM with STFM.

Concerning the compatibility of the API recommendations, we can easily find that both STFM and MTM reduce CPB@N compared with MTFM. The situation is quite obvious in the top-5 recommendations where the performance degrades about 5 percent. The situation becomes less-obvious with the increment of N. First of all, CPB@N is not an order-aware measure, its value tends to be a constant with the increment of N; Second, it is better to capture co-invocation pattern of APIs by transforming the task of API recommendation into the problem of multi-label classification, so as to favour those highly co-occurred APIs. Third, both the strategies of multi-model fusion and multi-task learning enable those relevant and compatible APIs to get highly-ranked synchronously.

3.5 Performance Comparison (RQ3 & RQ4)

In this section, the performance of MTFM is compared with the selected baseline methods. Experimental results on the four performance indicators are shown in Figs. 6a, 6b, 6c, and 6d.

For the baseline models, we found that the CF model performed the worst, especially in NDCG and MAP indicators. CF uses the bag of words model and the vector space model to modeling the document of mashup. It depends on relatively rich document information to identify mashups with similar requirements. Unfortunately, many mashup documents in the ProgrammableWeb dataset are not informative,

TABLE 4
Performance Comparison of Ablation Models w.r.t MTFM on the Task of Mashup Category Judgment

Methods	NDCG@5	NDCG@10	Recall@5	Recall@10
MTM	0.7152	0.7185	0.5752	0.6549
MTFM	**0.7757**	**0.7691**	**0.6267**	**0.7064**

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

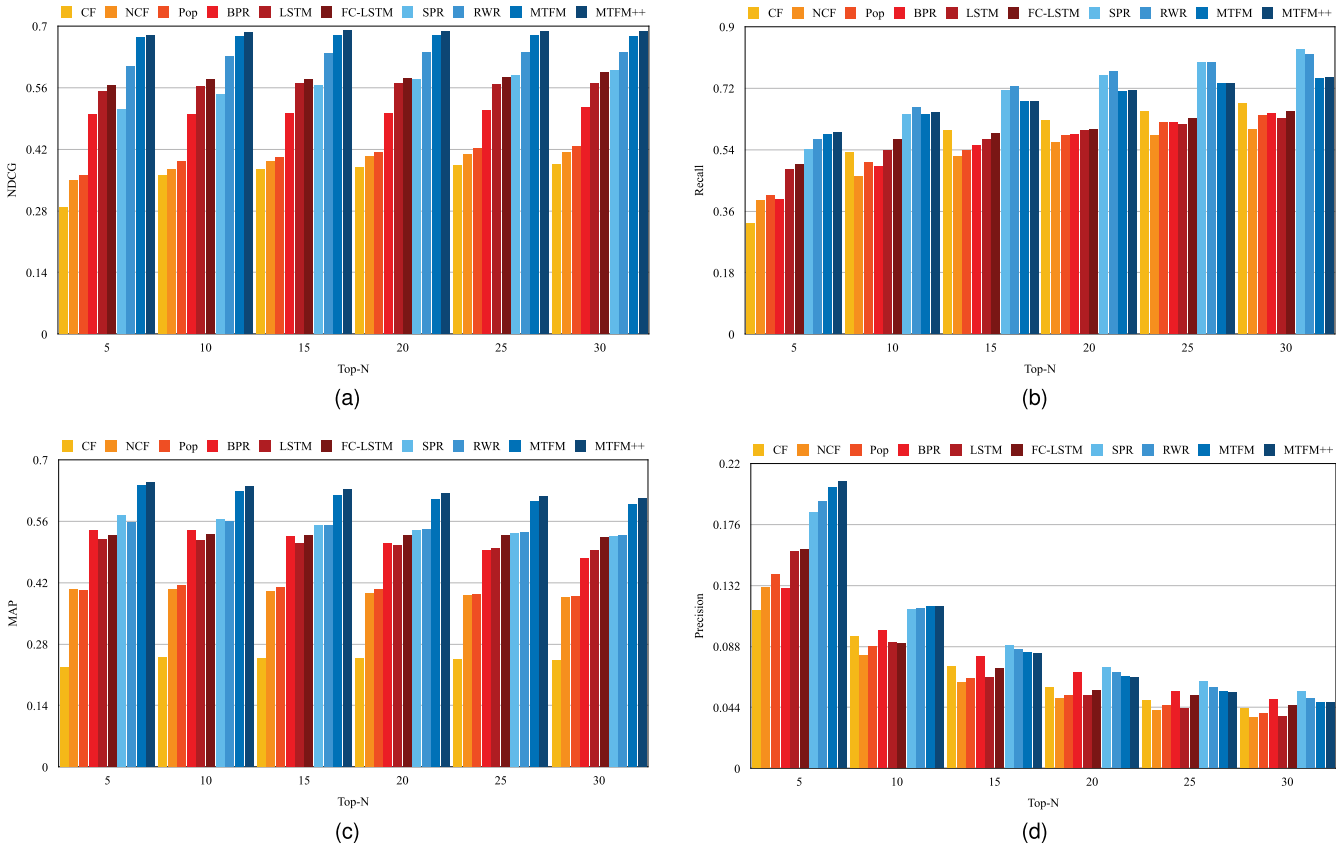

Fig. 6. Performance comparisons of all methods listed. (a) shows the NDCG@N results. (b) shows the Recall@N results. (c) shows the MAP@N results. And (d) shows the Precision@N results.

which makes CF unable to run robustly. NCF performed better than CF, but not as well as we expected. MTFM uses the same FIC and SC as NCF. However, NCF does not have U_{SC} in U_{MMF} and the auxiliary task. Also, NCF has not the same loss function as ours, which leads to a big performance gap. Simple as pop is, it performs slightly better than NCF.

For BPR, LSTM, FC-LSTM, and SPR, they achieve comparable accuracy. The advantage of BPR lies in direct ranking optimization. The advantage of LSTM lies in the superior encoding structure of bi-directional LSTM with an attention mechanism. The advantage of SPR is that the hidden topic layer can overcome the disadvantages of the bag of words model, and semantically connect the requirements with the APIs even though the given documents are not informative. Concerning FC-LSTM, it is specially designed based on LSTM with two attention mechanisms and thus consistently outperforms LSTM. FC-LSTM underperforms SPR in terms of Precision, Recall, and MAP, but outperforms SPR in terms of NDCG. Regarding RWR, it is the most competitive baseline, in particular on the recall rate. However, RWR describes the requirements of mashups with tag combinations and can not process the textual requirement. This increases the difficulty of operation for users as they have to describe their needs accurately and completely.

Compared with all the baseline models, MTFM has the advantages of both CF-based methods and content-based methods, also is leveraged by multi-task learning. MTFM outperforms them significantly on all the accuracy indicators. The trend is more significant when top-5 and top-10 recommendations are considered. Because each mashup uses only a small number of APIs, averaging less than two, it makes sense to look at the head of the recommendation list. As N ranges from 5 to 10, MTFM improves the baseline methods by at least 6.3-4.4 percent in NDCG, 8.4-6.8 percent in MAP, 1.0-0.2 percent in Precision and 1.4 percent in Recall. By leveraging on the textual features and quality features of Web APIs, MTFM++ improves MTFM by at least 1.5 percent in terms of all metrics of accuracy. This also shows that MTFM has scalability and potential for further improvement.

Finally, we perform compatibility comparison of all methods and show the results in Table 5. Different from the performance on accuracy, all the methods can be clearly classified into three grades. CF and NCF perform the worst in terms of CPB@5 and CPB@10, and fail to compare with other methods. Pop, LSTM, FC-LSTM and BPR can be divided into one gear. Pop has no advantage on the metrics of accuracy as compared with BPR, LSTM and FC-LSTM, but its performance of compatibility is not bad. The main reason is that some popular APIs (such as Google-Maps, Twitter, Youtube, Facebook, etc.) co-participate in many mashup applications , so they have better compatibility than those less-popular (or long-tail) APIs. SPR, RWR, MTFM and MTFM++ are in the best grades, which are obviously better than other grades. Although SPR, RWR and MTFM are based on different principles, their common point is to simultaneously model the relationships of Mashup-API and co-invocation of APIs to capture potential patterns, so they get comparable performance. Of course,

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

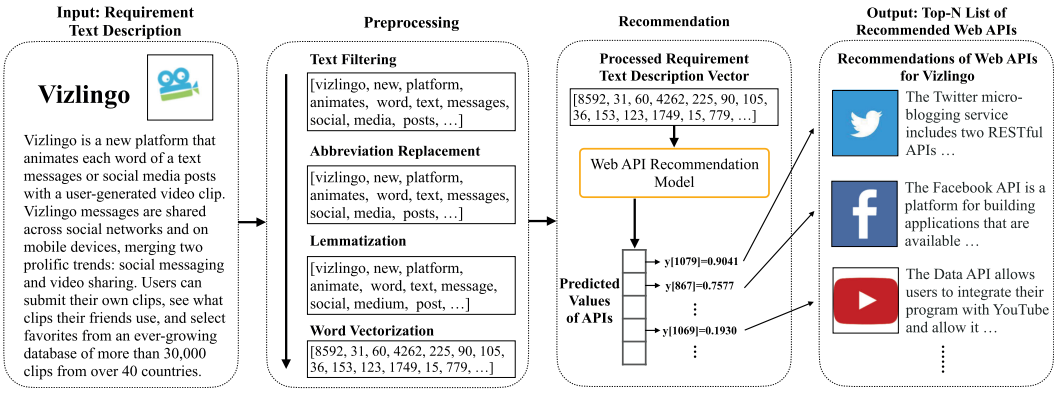

Fig. 7. A running example on Web API recommendation for *Vizlingo*.

MTFM/MTFM++ is still superior to all baselines in term of CPB@5.

3.6 Case-Study Analysis (RQ3 & RQ4)

In this section, we will show a running example and some actual cases of the API recommendation to make a qualitative comparison of the selected methods.

3.6.1 Running Example

Fig. 7 demonstrates the running example on Web API recommendation for *Vizlingo*. The textual description of Vizlingo is still assigned as the requirement. After preprocessing, the textual requirement will be converted to a sequence of word identifiers. In MTFM(++), each word identifier in the sequence will be mapped to a trainable 300d vector; these vectors will be compressed into a dense vector through semantic component, and then interact with the feature embedding of Web APIs; After several steps of *forward propagation*, the relevance score between each candidate API and the requirement, i.e., $y[a]$ (a is the index of API), is output in the task layer. Finally, top-N items are selected as the recommendation list after sorting all candidates by $y[a]$. For example, for the APIs of Twitter, Facebook, and Youtube, there are y[1079] = 0.9041, y[867] = 0.7577, y[1069] = 0.1930.

Note that our recommender system can be regarded as an ad-hoc IR system where users can refine the recommendation results incrementally by improving their descriptions of queries. Suppose that we use "*Vizlingo is a new platform that animates each word of text messages posts with a user-generated video clip*" as a query, the top-5 recommendations are *Youtube*, *Twitter*, Wikipedia, Google-Search, *Facebook*. The ranking performance is not as good as that of example, where all the text of description is used as the query.

3.6.2 Cases on API Recommendation

Table 6 shows the top five suggestions for four target mashups: *Vizlingo*, *Stweet*, *Recordlective* and *Arab-World-Weather*, which cover different types of applications.

Recall the running example for *Vizlingo*, both MTFM and MTFM++ perfectly hit the three target APIs in top-3 recommendations. It is a significant improvement compared to other baseline models even they are excellent with top-5 recommendations.

Stweet is a mix of street and twitt, offering a real new way to discover geolocalised twitts from Twitter on a Google Street View panorama. Stweet is an artistic project dealing with the appropriation of photographic and real-time data from the Web.[7] Most of the recommended methods can predict *Google-Maps* and *Twitter*, reaching an excellent level. But CF, NCF, Pop, and BPR just reach a pass line level.

Recordlective (RL) combines various sources to provide music fans the ability to stream music albums from start to finish.[8] Since the APIs involved are relatively rare, most methods only hit a single option. BPR, SPR and RWR can hit *Last.fm* and *Youtube* at the same time. MTFM and MTFM++ perform well in the case, hit all the target APIs, including *Musicbrainz*.

Arab-World-Weather (AWW) provides up-to-date weather forecasts of over all the major cities in the Arab World using Google maps, Google Chart, Panoramio, Open Weather Map, and World Weather Online APIs.[9] This mashup includes five different APIs, it is a challenge for all the recommendation models. The baseline model failed in this case. From CF to SPR, they only hit one *Google-Maps* to reach the passing level, while only powerful model RWR and our MTFM, MTFM++ hits 3 targets to reach the excellent level.

3.6.3 Cases on Mashup Category Judgement

The prediction performances of MTFM and MTFM++ on the task of mashup category judgment are shown in Table 7.

As for the mashups of *Stweet* and *AWW (Arab-World-Weather)*, MTFM and MTFM++ get a hit ratio of 100 percent on the category prediction. In the rest of the tasks, both models can achieve satisfactory performance. As for the mashups of *RL (Recordlective)* and *Vizlingo*, MTFM can also hit two given top-4/5 suggestions. MTFM++ makes progress in predicting categories of *Vizlingo*, and hits three items.

Totally, the prediction performance of MTFM/MTFM++ in the task of mashup category judgment is close to that of Web API recommendation.

3.7 Further Summarization From Experiments

By summing up the above experimental results, we can further draw some useful conclusions which also reveal the importance of our proposal compared with other approaches.

7. https://www.programmableWeb.com/mashup/stweet
8. https://www.programmableWeb.com/mashup/recordlective
9. https://www.programmableweb.com/mashup/arab-world-weather

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 5
Compatibility Comparison of all Methods

Methods	CPB@5	CPB@10	Methods	CPB@5	CPB@10
CF	0.6504	0.5496	BPR	0.8659	0.8655
NCF	0.6777	0.5511	SPR	0.9104	*0.8773*
Pop	0.8190	0.8015	RWR	*0.9211*	0.8674
LSTM	0.8400	0.8278	MTFM	**0.9236**	**0.8725**
FC-LSTM	0.8552	0.8385	MTFM++	**0.9301**	**0.8819**

First, the fusion of multiple models or features can overcome the drawbacks of a single model and achieve the goal of multi-win. Second, deep neural networks have better modeling ability and scalability, and can deal with the requirements of multi-model fusion, multi-task learning, multi-feature representation using an end-to-end solution. Third, natural text is used to describe the requirements, which provides a controllable way of interaction, so that users can incrementally refine the recommendation results. Finally, it is beneficial to improve the recommendation accuracy and compatibility by transforming the task of API recommendation into the problem of multi-label classification.

4 Related Works

The purpose of Web API recommendation is to propose a suitable subset of candidate APIs for users according to mashup requirements, to reduce the burden of manual

TABLE 6
Case-Study Analysis on API Recommendation Where the APIs Get Hit are Marked in **bold font**

Mashup	Method	Top 5 Recommendations of APIs
Vizlingo	CF	Twilio-Sms, **Facebook**, Microsoft-Bing-Maps, **Youtube**, Amazon-Product-Advertising
	NCF	Spore, Google-Maps, **Twitter**, Windshieldrepair-Tech-Auto-Glass, Google-Translate
	Pop	Google-Maps, **Youtube**, **Twitter**, Twilio, Flickr
	BPR	**Twitter**, **Facebook**, Google-Maps, Flickr, **Youtube**
	LSTM	Google-Maps, **Twitter**, **Youtube**, Flickr, **Facebook**
	FC-LSTM	**Youtube**, Google-Maps, **Twitter**, Flickr, **Facebook**
	SPR	Google-Maps, **Twitter**, **Youtube**, **Facebook**, Flickr
	RWR	Google-Maps, **Twitter**, **Youtube**, **Facebook**, Flickr
	MTFM	**Twitter**, **Facebook**, **Youtube**, Microsoft-Bing-Maps, Amazon-Product-Advertising
	MTFM++	**Twitter**, **Facebook**, **Youtube**, Google-App-Engine, Foursquare
	GT	**Facebook** , **Twitter** , **Youtube**
Stweet	CF	Youtube, **Twitter**, Flickr, Facebook, Last.fm
	NCF	**Google-Maps**, Shizzow, Google-Search, Yipit, Last.fm
	Pop	**Google-Maps**, Youtube, Flickr, Amazon-Product-Advertising, **Twitter**
	BPR	**Google-Maps**, **Twitter**, Facebook, Flickr, Youtube
	LSTM	**Google-Maps**, **Twitter**, Youtube, Flickr, Facebook
	FC-LSTM	**Google-Maps**, **Twitter**, Facebook, Youtube, Flickr
	SPR	**Google-Maps**, **Twitter**, Youtube, Flickr, Facebook
	RWR	**Google-Maps**, **Twitter**, Facebook, Flickr, Youtube
	MTFM	**Twitter**, **Google-Maps**, Yahoo-Maps, Flickr, Yahoo-Geocoding
	MTFM++	**Twitter**, **Google-Maps**, Yahoo-Maps, Yahoo-Geocoding, Google-Chart
	GT	**Google-Maps**, **Twitter**
RL	CF	Amazon-Product-Advertising, Youtube, Amazon-S3, Twitter, Flickr
	NCF	Amazon-Product-Advertising, Longurl, Venteria, Google-Maps, Google-Earth
	Pop	Google-Maps, Twitter, Facebook, **Youtube**, Twilio
	BPR	**Youtube**, Google-Maps, Flickr, Amazon-Product-Advertising, **Last.fm**
	LSTM	Google-Maps, Twitter, **Youtube**, Flickr, Facebook
	FC-LSTM	**Youtube**, Google-Maps, Facebook, Twitter, Twilio
	SPR	**Youtube**, Google-Maps, **Last.fm**, Twitter, Flickr
	RWR	Google-Maps, Twitter, **Youtube**, Flickr, **Last.fm**
	MTFM	**Last.fm**, **Youtube**, Spotify-Metadata, **Musicbrainz**, Lyricwiki
	MTFM++	**Last.fm**, Facebook, Twitter, **Musicbrainz**, **Youtube**
	GT	**Last.fm**, **Musicbrainzs**, **Youtube**
AWW	CF	Flickr, Amazon-Product-Advertising, Youtube, Twilio, **Google-Maps**
	NCF	Flickr, **Google-Maps**, Social-Mention, Google-Analytics-Managment, Foursquare
	Pop	**Google-Maps**, Twitter, Youtube, Flickr, Facebook
	BPR	Twilio, Twilio-Sms, Twitter, Google-App-Engine, **Google-Maps**
	LSTM	**Google-Maps**, Twitter, Youtube, Flickr, Facebook
	FC-LSTM	**Google-Maps**, Twitter, Facebook, Youtube, Flickr
	SPR	**Google-Maps**, Twitter, Youtube, Flickr, Facebook
	RWR	**Google-Maps**, **Openweathermap**, Sendy, **Google-Chart**, Viator
	MTFM	Weatherbug, **Google-Maps**, **World-Weather-Online**, **Openweathermap**, Forecast
	MTFM++	**World-Weather-Online**, **Google-Maps**, Weatherbug, Yahoo-Traffic, **Openweathermap**
	GT	**Google-Chart**, **Google-Maps**, **Openweathermap**, **Panoramio**, **World-Weather-Online**

GT means Ground Truth.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 7
Case-Study Analysis on Mashup Category Judgment Where the Categories Get Hit are Marked in **bold font**

Mashup	Method	Top 4 or 5 Predictions of Categories
Vizlingo	MTFM	**Social**, **Messaging**, Chat, Mobile, Notifications
	MTFM++	**Social**, Mobile, **Video**, **Messaging**, Reference
	GT	**Social**, **Messaging**, **Tools**, **Video**
Stweet	MTFM	**Mapping**, Visualizations, **Blogging**, Social
	MTFM++	**Mapping**, **Blogging**, Visualizations, Reference
	GT	**Mapping**, **Blogging**
RL	MTFM	**Music**, **Search**, Entertainment, Lyrics, Video
	MTFM++	**Music**, Social, Shopping, Reference, **Search**
	GT	**Aggregation**, **Music**, **Search**, **Streaming**, **Tools**
AWW	MTFM	**Weather**, **Geography**, Mapping, Visualizations
	MTFM++	**Weather**, **Geography**, Mapping, Marketplace
	GT	**Weather**, **Geography**

selection. Here, we divide the related work into four parts according to their principles and emphasis, namely, collaborative filtering-based methods, content-based methods, deep learning methods, and service mesh and composition.

4.1 Collaborative Filtering Based Methods

Collaborative filtering utilizes like-minded objects to make recommendations, either in user-based CF or in item-based CF, as well as model-based CF (e.g., matrix factorization). CSCF [38] was proposed to rank and recommend Mashup services by combining the content similarity between user history records and Mashup services, and the estimated QoS value of active users to target service by using both user-based and service-based collaborative filtering. Chen *et al.* [42] combined user-based CF with item-based CF and proposed a Web service QoS prediction method, which helps users discover suitable Web services by analyzing QoS information. Similarly, Wu *et al.* [43] proposed a context-sensitive matrix-factorization approach to make collaborative QoS prediction by considering the complexity of service invocations and modeling the interactions of users-to-services and environment-to-environment simultaneously. Different from those efforts that concentrate on the QoS prediction approach for service recommendation[38], [42], [43], Rahman *et al.* [10] used a two-level topic model for clustering Mashup services and then proposed a matrix factorization based API recommendation method for Mashup services. By analyzing the common calling patterns of Web APIs and discovering potential associations between them, Yao *et al.* [11] proposed a probabilistic matrix factorization method with implicit correlation regularization to identify the most appropriate APIs for composition task and enhance the diversity of recommendations.

CF-based methods can share the experience of similar mashups, and can also achieve certain results in the case of sparse data, but do not give enough attention to the requirements themselves. That is to say, they rarely pay attention to the matching between Web API and mashup requirements. In contrast, we have treated the Web API recommendation as a task of multi-label classification and developed a complete end-to-end framework to realize the matching between Web API and mashup requirements.

4.2 Content Based Methods

Beyond collaborative filtering, another kind of method for Web API recommendation focuses on mining the relationship between requirements of mashups and Web APIs, and directly suggests those Web APIs close to the requirements. Also, various technologies, e.g., topic modeling [12], [14] and link analysis [15], [19], in combination with various features, e.g., texts, tags, topics, and popularity are exploited to enhance the accuracy of recommendations [23], [24], [25], [26]. After modeling the service description based on the topic model, Xia *et al.* [12] proposed clustering the services with categories as the center and then combined collaborative filtering to predict the ranking order of services in each category. Experimental results confirmed that category information plays a role that cannot be ignored in reducing service recommendation redundancy, decomposing mashup requirements, and improving the diversity of service recommendations. Li *et al.* [25] used the relational topic model to model the tags and topical information of mashup and Web APIs, and calculated the similarity between Web APIs and mashup respectively. They also used Web API call time and category information to derive their popularity, and finally exploited factorization machines to predict and recommend top-related Web APIs with these hand-crafted features. Unlike the above methods, Bao *et al.* [14] proposed a fine-grained API link prediction approach to support mashup recommendation. Especially, the authors claimed that different attributes may contribute differently to a service linkage. Instead of training a model for all attributes as a whole, a novel approach was presented to simultaneously train separate models for individual attributes. Meanwhile, a latent attribute modeling method was developed to reveal context-aware attribute distribution. To leverage mashup descriptions and structures, and bridge the vocabulary gap between mashup developers and Web APIs, Zhong *et al.* [18] proposed to jointly model mashup descriptions and Web APIs using the author-topic model. They also proposed a novel two-phase service recommendation framework to facilitate mashup creation based on reconstructing service profiles. Additionally, Cao *et al.* [16] used the relationship between Mashup and services to develop a two-level topic model and tap potentially useful topics to improve the accuracy of service clustering. Based on the clustering results of Mashups, they designed a Web API recommendation algorithm based on collaborative filtering, using the implicit co-invocation relationship between Web APIs inferred from the invocation history between Mashups clusters and the corresponding Web APIs, to recommend diverse Web APIs for each cluster of Mashups.

The main problem of content-based methods is a pipeline architecture. This process requires a lot of human participation to extract effective features, especially for text data. Compared with these works, we use a deep neural network as the modeling basis to build a complete end-to-end recommendation framework. CNN is used to endow the recommendation model with the ability of automatic feature extraction to minimize manual intervention. The end-to-end framework also allows incremental training of the model, avoiding the disadvantage that the topic model needs all datasets for each training.

4.3 Deep Learning-Based Methods

With the rapid development of deep learning, it has become a trend to deliver neural networks in the API recommendations. Based on text mining, Xue *et al.* [20] proposed ranking the generated API recommendation list and recommended it to users based on whether the given APIs meet

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

quantitative indicators. By combining collaborative filtering and description text, Xiong *et al.* [21] proposed a novel hybrid method based on deep learning for service recommendation. The calls and functions between the mashup and the service are seamlessly integrated into a deep neural network, which can be used to characterize the complex relationship between the mashup and the services. Shi *et al.* [22] proposed a service recommendation method based on a deep model. Specifically, they first extend the service description at the sentence level based on a probabilistic topic model, which learns representations of words, sentences, and described topics hierarchically. Then an LSTM model with a functional attention mechanism and contextual attention mechanism of content was proposed to help recommend the most suitable APIs.

These works have proved that deep learning technology can help to develop more efficient API recommendation models. However, none of them focus on multi-model fusion and multi-task learning. In contrast, we have explored these aspects and obtained positive results. These are the essential differences between our work and their works.

4.4 Service Mesh and Composition

Service composition refers to the ability to combine Web services into a business process. Traditional service composition emphasizes orchestration or choreography of services [44]. As a special case of service compositions [44], Mashup refers to an ad hoc composition technology of Web applications that allows users to aggregate existing Web resources to create entirely new services [45]. Compared to traditional "developer-centric" composition technologies, e.g., BPEI and WSCI, mashup provides a flexible and easy-of-use way for service composition on Web [46].

Service mesh is a configurable, low-latency infrastructure layer designed to handle a high volume of network-based interprocess communication among application infrastructure services using APIs [47]. Service mesh ensures that communication among containerized and often ephemeral application infrastructure services is fast, reliable, and secure. Both service mesh and Mashup are intending to enable service reuse in composition. As the number of APIs exceeds the limits that can be manually discovered and composed, it is meaningful to develop intelligent solutions. At this point, our work can be seen as a small step towards intelligent service discovery and selection.

5 Conclusion

To address the limits of mashup-oriented Web API recommendation, we have proposed a novel deep model, which extracts the collaborative information of mashup through the feature interaction component, and models descriptive information through the semantic component. Based on the fusion feature, we take the mashup category judgment as an auxiliary task to improve the performance of the main task of API recommendation through inductive bias. The experimental results demonstrate that multi-task learning in combination with multiple information sources can effectively improve the accuracy and compatibility of the API recommendations, and exploiting the metadata and quality features of Web APIs can bring extra benefits.

However, current AI technology cannot fully realize service discovery and composition, and more research efforts are needed. First, more powerful neural network models, such as BERT and Capsulate network can be taken in the semantic component to strengthen the effects of text modeling. Second, we can initialize the embeddings of Web APIs with more informative resources. Additionally, we can continue to explore the possibility of multi-task models in the task of Web API recommendations, such as introducing pos tasks in the NLP field.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grants 61962061, 61562090, U1802271, in part by the Yunnan Provincial Foundation for Leaders of Disciplines in Science and Technology under Grant 202005AC160005, Top Young Talents of "Ten Thousand Plan" in Yunnan Province YNWR-QNBJ-2019-188, the Program for Excellent Young Talents of Yunnan University.

References

[1] W. Tan, Y. Fan, A. Ghoneim, M. A. Hossain, and S. Dustdar, "From the service-oriented architecture to the web API economy," *IEEE Internet Comput.*, vol. 20, no. 4, pp. 64–68, Jul./Aug. 2016.

[2] J. Yu, B. Benatallah, F. Casati, and F. Daniel, "Understanding mashup development," *IEEE Internet Comput.*, vol. 12, no. 5, pp. 44–52, Sep./Oct. 2008.

[3] S. Roy Chowdhury, F. Daniel, and F. Casati, "Recommendation and weaving of reusable mashup model patterns for assisted development," *ACM Trans. Internet Technol.*, vol. 14, no. 2/3, Oct. 2014, Art. no. 21.

[4] F. Daniel, M. Matera, and M. Weiss, "Next in mashup development: User-created apps on the web," *IT Professional*, vol. 13, no. 5, pp. 22–29, 2011.

[5] A. T. Nguyen *et al.*, "API code recommendation using statistical learning from fine-grained changes," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2016, pp. 511–522.

[6] Q. He, B. Li, F. Chen, J. Grundy, X. Xia, and Y. Yang, "Diversified third-party library prediction for mobile app development," *IEEE Trans. Softw. Eng.*, to be published, doi: 10.1109/TSE.2020.2982154.

[7] Q. He, R. Zhou, X. Zhang, Y. Wang, D. Ye, F. Chen, J. C. Grundy, and Y. Yang, "Keyword search for building service-based systems," *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 658–674, Jul. 2017.

[8] P. Cremonesi, M. Picozzi, and M. Matera, "A comparison of recommender systems for mashup composition," in *Proc. 3rd Int. Workshop Recommendation Syst. Softw. Eng.*, 2012, pp. 54–58.

[9] J. Cao, W. Xu, L. Hu, J. Wang, and M. Li, "A social-aware service recommendation approach for mashup creation," *Int. J. Web Serv. Res.*, vol. 10, no. 1, pp. 53–72, 2013.

[10] M. M. Rahman, X. F. Liu, and B. Cao, "Web API recommendation for mashup development using matrix factorization on integrated content and network-based service clustering," in *Proc. IEEE Int. Conf. Services Comput.*, 2017, pp. 225–232.

[11] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with API co-invocations," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 502–515, Mar./Apr. 2021.

[12] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.

[13] Q. Gu, J. Cao, and Q. Peng, "Service package recommendation for mashup creation via mashup textual description mining," in *Proc. IEEE Int. Conf. Web Services*, 2016, pp. 452–459.

[14] Q. Bao *et al.*, "A fine-grained API link prediction approach supporting mashup recommendation," in *Proc. IEEE Int. Conf. Web Services*, 2017, pp. 220–228.

[15] D. Bianchini, V. D. Antonellis, and M. Melchiori, "WISeR: A multi-dimensional framework for searching and ranking web APIs," *ACM Trans. Web*, vol. 11, no. 3, pp. 19:1–19:32, 2017.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.

[16] B. Cao, X. F. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIs recommendation for mashup development," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.
[17] L. Qi *et al.*, "Finding all you need: Web APIs recommendation in web of things through keywords search," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 5, pp. 1063–1072, Oct. 2019.
[18] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.
[19] X. Wang, H. Wu, and C. H. Hsu, "Mashup-oriented API recommendation via random walk on knowledge graph," *IEEE Access*, vol. 7, pp. 7651–7662, 2019.
[20] Q. Xue, L. Liu, W. Chen, and M. C. Chuah, "Automatic generation and recommendation for API mashups," in *Proc. 16th IEEE Int. Conf. Mach. Learn. Appl.*, 2017, vol. 2017-Decem, pp. 119–124.
[21] R. Xiong, J. Wang, N. Zhang, and Y. Ma, "Deep hybrid collaborative filtering for Web service recommendation," *Expert Syst. Appl.*, vol. 110, pp. 191–205, 2018.
[22] M. Shi, Y. Tang, and J. Liu, "Functional and contextual attention-based LSTM for service recommendation in mashup creation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 5, pp. 1077–1090, May 2019.
[23] Y. Zhong, Y. Fan, K. Huang, W. Tan, and J. Zhang, "Time-aware service recommendation for mashup creation in an evolving service ecosystem," in *Proc. IEEE Int. Conf. Web Services*, 2014, pp. 25–32.
[24] Z. Yu, R. K. Wong, and C. Chi, "Efficient role mining for context-aware service recommendation using a high-performance cluster," *IEEE Trans. Services Comput.*, vol. 10, no. 6, pp. 914–926, Nov./Dec. 2017.
[25] H. Li, J. Liu, B. Cao, M. Tang, X. F. Liu, and B. Li, "Integrating tag, topic, co-occurrence, and popularity to recommend web APIs for mashup creation," in *Proc. IEEE Int. Conf. Services Comput.*, 2017, pp. 84–91.
[26] K. K. Fletcher, "A quality-aware web API recommender system for mashup development," in *Proc. 16th Int. Conf. Services Comput.*, 2019, pp. 1–15.
[27] L. Qi, Q. He, F. Chen, X. Zhang, W. Dou, and Q. Ni, "Data-driven web APIs recommendation for building web applications," *IEEE Trans. Big Data*, to be published, doi: 10.1109/TBDATA.2020.2975587.
[28] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing," *IEEE Comput. Intell. Mag.*, vol. 13, no. 3, pp. 55–75, Aug. 2018.
[29] M. Imran, S. Soi, F. Kling, F. Daniel, F. Casati, and M. Marchese, "On the systematic development of domain-specific mashup tools for end users," in *Proc. Int. Conf. Web Eng.*, 2012, pp. 291–298.
[30] R. Johnson and T. Zhang, "Effective use of word order for text categorization with convolutional neural networks," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2015, pp. 103–112.
[31] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1746–1751.
[32] S. Rendle, "Factorization machines with libFM," *ACM Trans. Intell. Syst. Technol.*, vol. 3, no. 3, pp. 57:1–57:22, 2012.
[33] T. Baltrušaitis, C. Ahuja, and L. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 2, pp. 423–443, Feb. 2019.
[34] S. Ruder, "An overview of multi-task learning in deep neural networks," *CoRR*, 2017. [Online]. Available: http://arxiv.org/abs/1706.05098
[35] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 8, pp. 1819–1837, Aug. 2014.
[36] C. Cappiello, F. Daniel, and M. Matera, "A quality model for mashup components," in *Proc. 9th Int. Conf. Web Eng.*, 2009, pp. 236–250.
[37] M. Ge, C. Delgado-Battenfeld, and D. Jannach, "Beyond accuracy: Evaluating recommender systems by coverage and serendipity," in *Proc. ACM Conf. Recommender Syst.*, 2010, pp. 257–260.
[38] B. Cao, M. Tang, and X. Huang, "CSCF: A mashup service recommendation approach based on content similarity and collaborative filtering," *Int. J. Grid Distrib. Comput.*, vol. 7, no. 2, pp. 163–172, 2014.
[39] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. World Wide Web Conf.*, 2017, pp. 173–182.
[40] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertainty Artif. Intell.*, 2009, pp. 452–461.
[41] J. Nowak, A. Taspinar, and R. Scherer, "LSTM recurrent neural networks for short text and sentiment classification," in *Proc. Int. Conf. Artif. Intell. Soft Comput.*, 2017, pp. 553–562.
[42] X. Chen, Z. Zheng, and M. R. Lyu, "QoS-aware web service recommendation via collaborative filtering," in *Web Services Foundations*. Berlin, Germany: Springer, 2014, pp. 563–588.
[43] H. Wu, K. Yue, B. Li, B. Zhang, and C. Hsu, "Collaborative QoS prediction with context-sensitive matrix factorization," *Future Gener. Comput. Syst.*, vol. 82, pp. 669–678, 2018.
[44] A. L. Lemos, F. Daniel, and B. Benatallah, "Web service composition: A survey of techniques and tools," *ACM Comput. Surv.*, vol. 48, no. 3, 2016, Art. no. 33.
[45] K. Sangsanit, W. Kurutach, and S. Phoomvuthisarn, "REST web service composition: A survey of automation and techniques," in *Proc. Int. Conf. Inf. Netw.*, 2018, pp. 116–121.
[46] X. Liu, Y. Hui, W. Sun, and H. Liang, "Towards service composition based on mashup," in *Proc. IEEE Int. Conf. Services Comput. Workshops*, 2007, pp. 332–339.
[47] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, "Service mesh: Challenges, state of the art, and future research opportunities," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng.*, 2019, pp. 1220–1225.

Hao Wu received the bachelor's degree in computer science from Zhengzhou University, China, in 2001, and the master's and PhD degrees in computer science from the Huazhong University of Science and Technology, China, in 2004 and 2007, respectively. Currently, he is an associate professor with the School of Information Science and Engineering, Yunnan University, China. He has published more than 50 papers in peer-reviewed international journals and conferences. He has co-authored two monographs published in World Scientific. His research interests include service computing and recommender systems.

Yunhao Duan received the BS degree in computer science from Yunnan University, Kunming, China, in 2018. He is currently a postgraduate student with Yunnan University, China. His current research interests mainly include deep learning and service computing.

Kun Yue received the BSc degree from Yunnan University, Kunming, China, in 2001, the MSc degree from Fudan University, Shanghai, China, in 2004, and the PhD degrees in computer science from Yunnan University, Kunming, China, in 2009. He is currently a professor with Yunnan University, Kunming, China. He has authored 80 papers in peer-reviewed journals and conferences, such as the *IEEE Transation Cybernetics*, *IEEE Transation Services Computing*, *IEEE Transation Multimedia*, *Information Sciences*, DASFAA, BigData. He has published two monographs in World Scientific. He has also served as a reviewer and PC member for many venues. His current research interests mainly include massive data analysis and uncertainty in artificial intelligence.

Lei Zhang received the BSc degree in computer science from Zhengzhou University, China, the MS degree in pattern recognition and intelligence system from the Chinese Academy of Sciences, China, and the PhD degree from Southeast University, China, in 2011. He was a research fellow with IPAM, UCLA, in 2008. He is currently an associate professor with the School of Electrical and Automation Engineering, Nanjing Normal University, China. His research interests include machine learning, human activity recognition, and computer vision.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:48:37 UTC from IEEE Xplore. Restrictions apply.